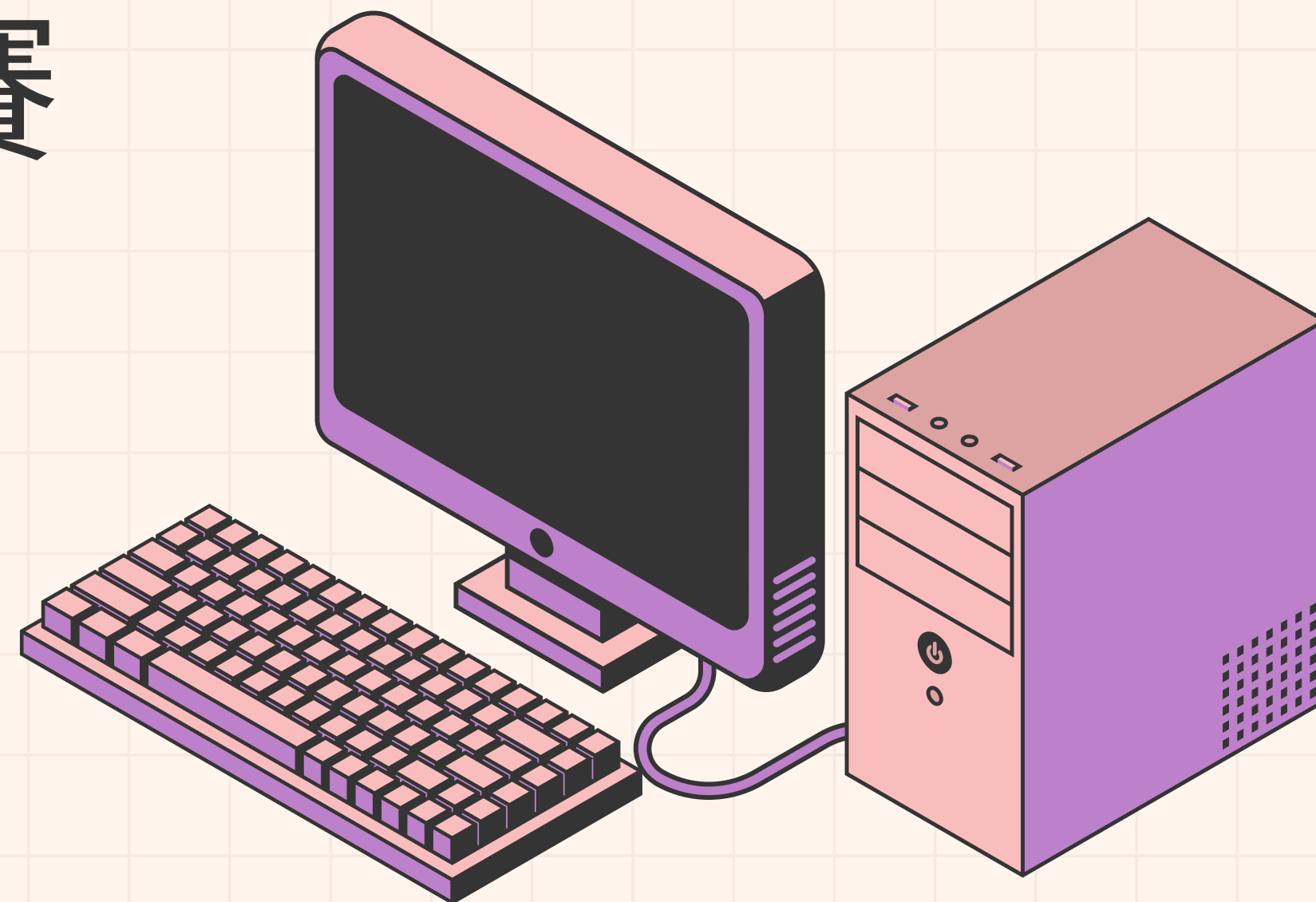


HPC-I 2026 模擬比賽



HPL 數據

	N	NB	P*Q	time	Gflops	state
1	36000	128	2*4	>36分	?	fail
2	24000	128	2*4	121.14	76.08	pass
3	30000	256	2*4	177.78	101.25	pass
4	26000	256	1*2	33.71	347.61	pass

數據1 --> 數據2：擴大 N (76.08 --> 101.25 GFLOPS)

在 RAM 足夠的前提下，N 設的越大，CPU 計算時間就越長，網路傳輸延遲的時間佔比就越低，最終測出來的 GFLOPS 就會越高

如果 N 設太大超過 RAM 的容量，電腦就會卡死

數據2 --> 數據3： 加大 BLOCK (101.25 --> 123.44 GFLOPS)

CPU 快取的速度比 RAM 快上百倍。如果 NB 設得剛剛好，就能完美塞滿 CPU 的 L2/L3 快取，減少cache miss

NB 太小：COMPULSORY CACHE MISS 暴增

SPATIAL LOCALITY 失效

NB 太大：CACHE THRASHING

數據3 --> 數據4: (123.44 --> 347.61 GFLOPS)

- 1.將 $P \times Q$ 從 2×4 改成 1×2
- 2.編譯時加入 `-march=native` 開啟 AVX2 向量指令集。
- 3.將矩陣反向縮小至 $N=26000$ 。

ICON

1. DEPENDENCIES



Compiled all libraries from source: ZLIB → HDF5 → NetCDF-C/Fortran → OpenBLAS → FYAML → XML2



Used a single custom prefix so nothing conflicted with system libraries



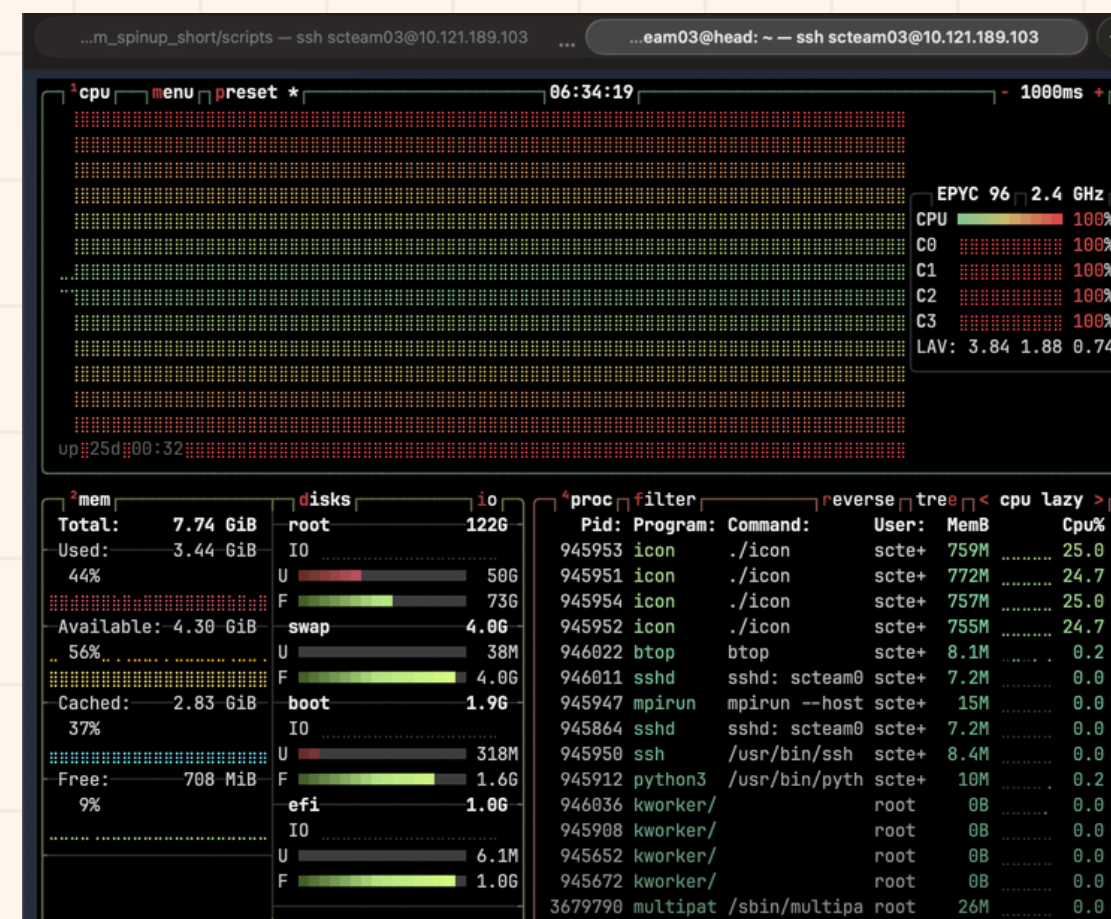
Ran git lfs install before cloning — without this, all binary files are silently broken stubs

Built ICON with `-march=native` for AVX2 SIMD on our CPUs

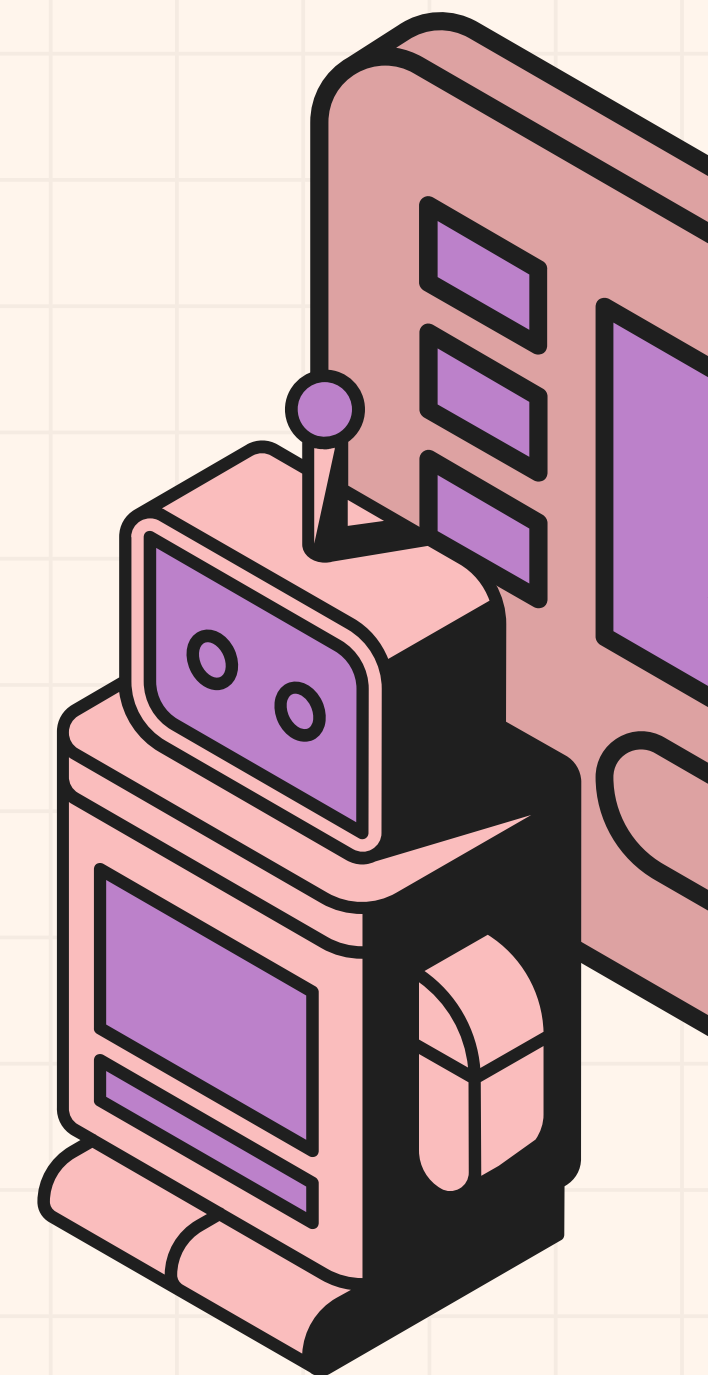
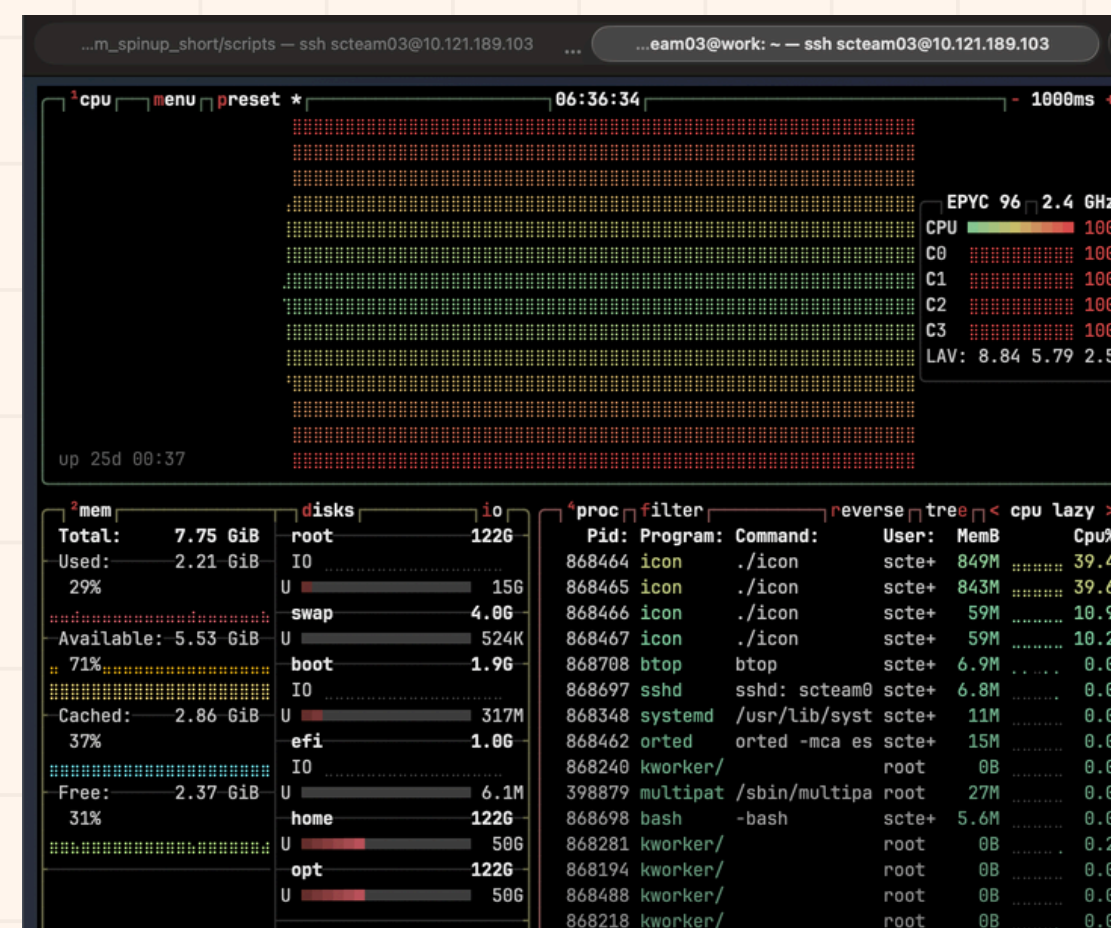
2. RUN SCRIPT MODIFICATIONS

- Original script was written for Slurm — our cluster has none
- Stubbed out scontrol() and set SLURM_JOBID=999999 to prevent crashes
- Replaced Slurm MPI launch with explicit: mpirun --host head:4,work:4 -n 8
- ICON initialized cleanly — restart file loaded, grid decomposed across 8 ranks

*Head node



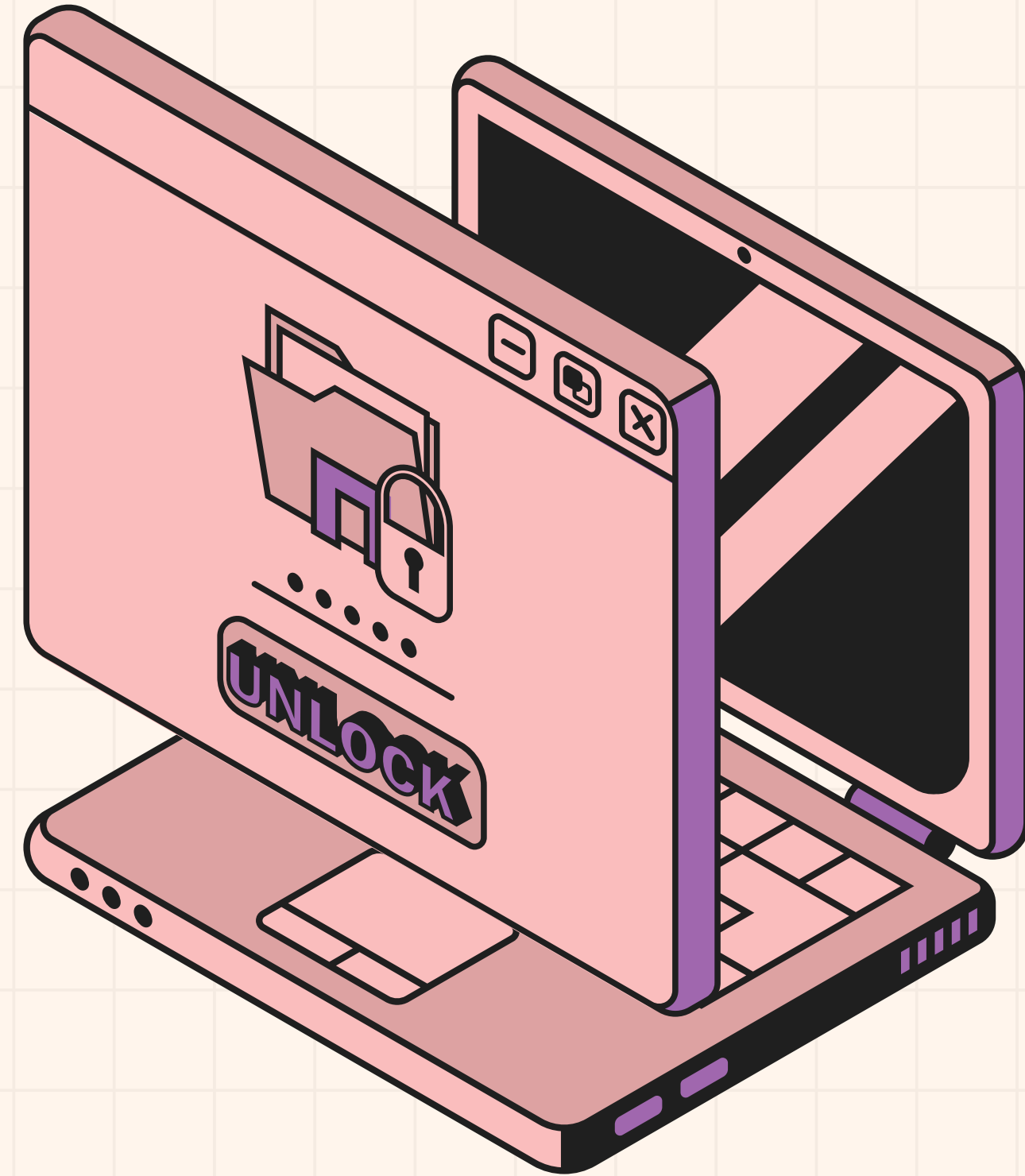
*Work node

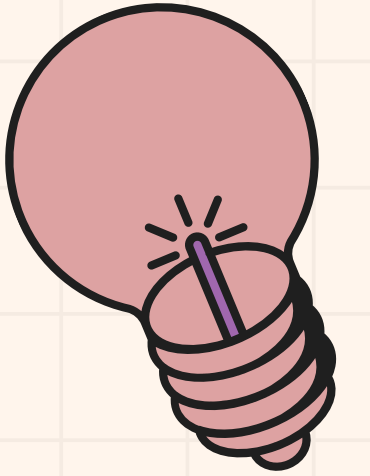




3. PROBLEMS

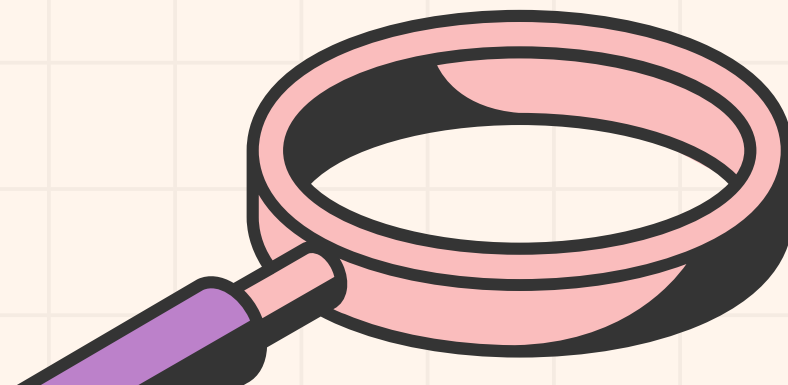
- Simulation started and timesteps were advancing — but wall-clock time ran out before the output interval was reached
- Root cause of time loss: two multi-node MPI bugs that were hard to diagnose
- Head node has two network interfaces → MPI registered on one, sent on the other → silent packet drops
- Fix: force TCP traffic to internal subnet only (OMPI_MCA_btl_tcp_if_include=10.21.21.0/24)
- ICON uses MPI one-sided (RMA) operations → osc/sm component broke when sm BTL was disabled → MPI_ERR_WIN on init
- Fix: OMPI_MCA_osc=pt2pt
- These two issues alone cost several hours before the actual simulation could even start





4. WHAT WE'D FIX TO GET OUTPUT

- Increase dynamics timestep from PT1M → PT4M
(4× fewer solver calls)
- nproma=32 already set for AVX2 vectorization
- Separate I/O procs (num_io_procs=2) so disk writes don't block compute



MLPerf Llama2-7B

From “it won’t run”
to 4.54x throughput

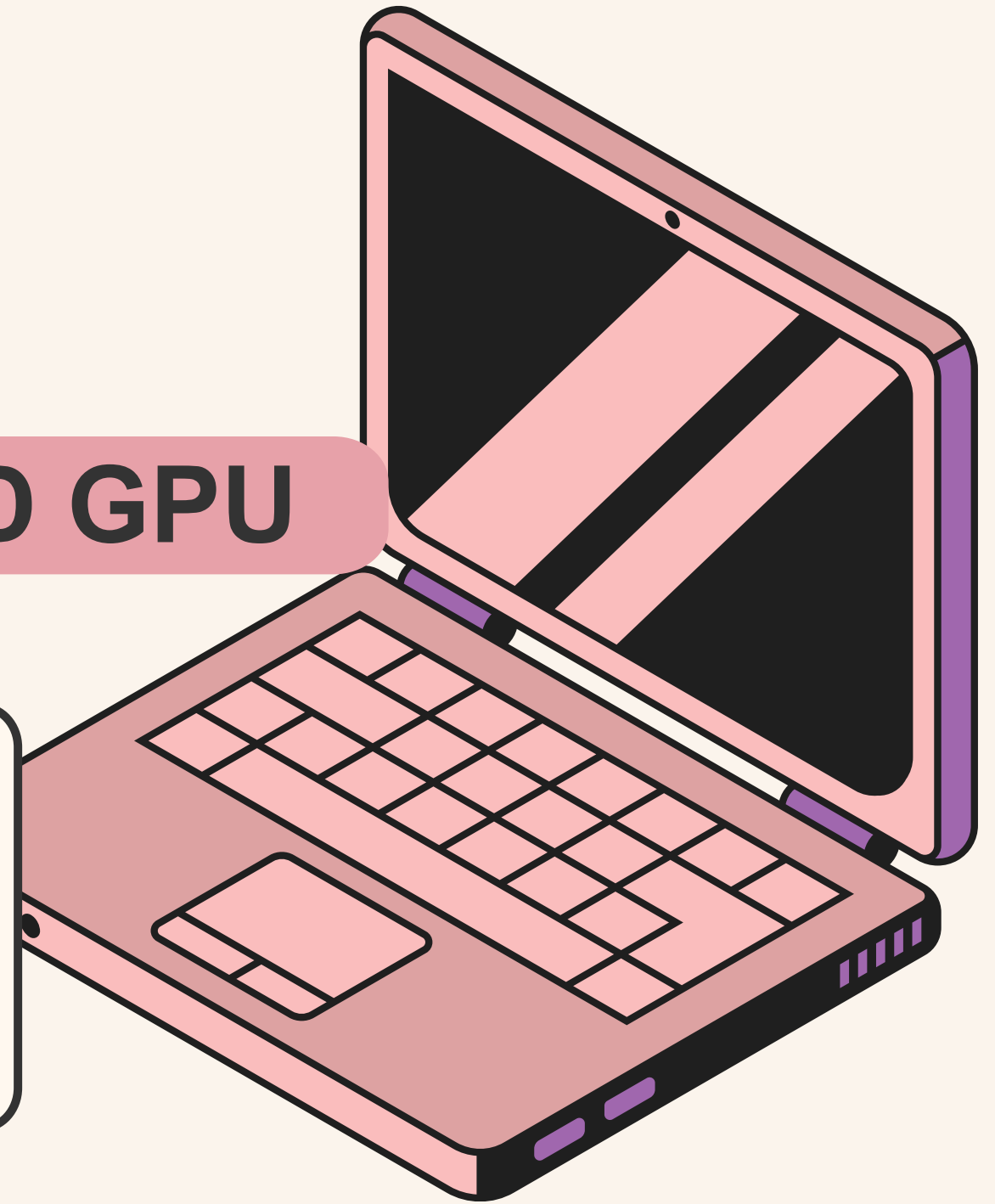
Offline · PerformanceOnly · single AMD GPU

262.6
baseline
tokens/s



1193.3
final tokens/s

4.54x
gain



🔗 GROUP 3 | MLPerf

**This was full-stack benchmark execution,
not just one config file.**

Any broken layer makes the tokens/s number meaningless.



**Crashes, stale configs, or multi-GPU
spillover would invalidate the story.**

GROUP 3 | MLPerf

I kept the official benchmark constraints fixed.

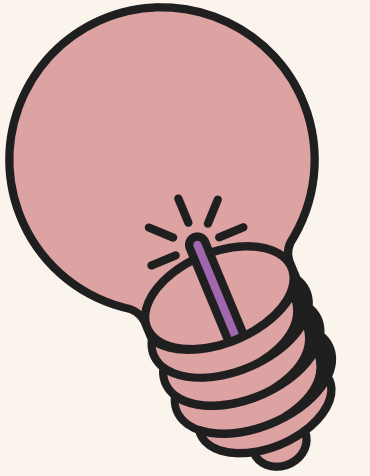
Fixed
Llama2-7B
Offline
PerformanceOnly
1 AMD GPU

Changed
Batching
Concurrency
GPU memory cap
vLLM switches



GROUP 3 | MLPerf

Most taxing blocker: storage



exhaustion.

No container meant no benchmark, no baseline, and no tuning.

most taxing

100%

/home full

99%

/ root full

315GB

/dev/shm free

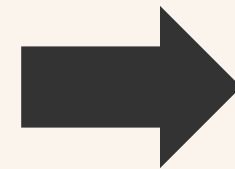
Observed failure: FATAL no space left on device

GROUP 3 | MLPerf

The workaround was to turn RAM into build scratch space.

**Redirect volatile
build pressure**

APPTAINER_TMPDIR
APPTAINER_CACHEDIR
/dev/shm/...



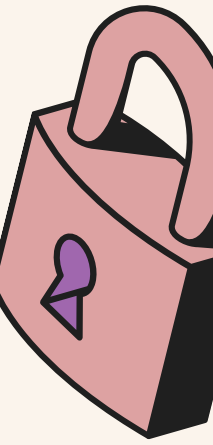
**Force hardcoded
paths**

symlink hidden
.apptainer_build
paths into RAM

**Result: an 8.0GB persistent .sif image was
generated.**

The hidden blocker was configuration shadowing.

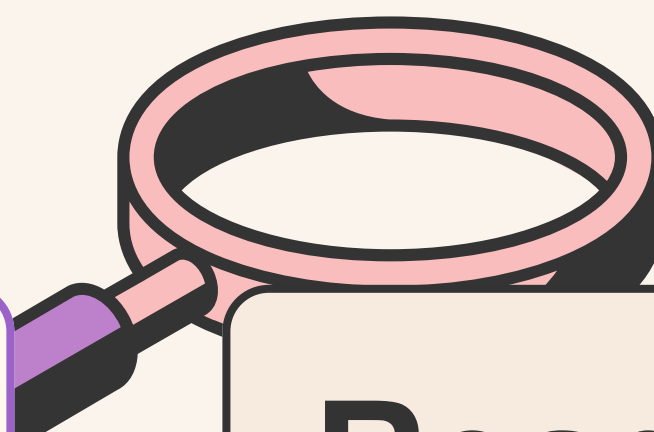
The job ran, but my YAML edits were invisible inside Apptainer.



This created a false sense of tuning until the mount was fixed.

GROUP 3 | MLPerf

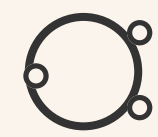
The fix was a bind mount plus a sanity check.



```
--bind host worksp  
/home/.../closed/AMD  
→  
/lab-mlperf-inference
```

Read YAML inside
cat from container
before Slurm run

After this, every tuning result became trustworthy.



GROUP 3 | MLPerf

The valid baseline exposed underutilization.

The default vLLM configuration was safe, not fast.

262.59

tokens/s

1.102

samples/

0.70

mem cap

8

seqs

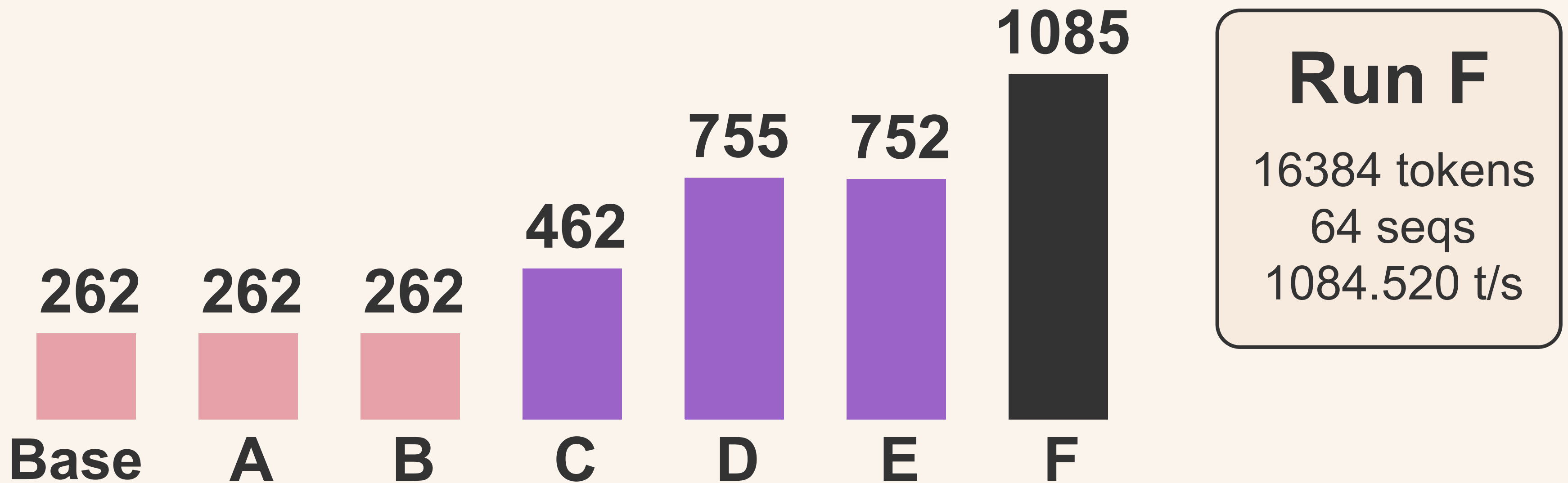
Hypothesis: Offline throughput needs more active sequences per GPU pass.



GROUP 3 | MLPerf

Sequence concurrency was the winning knob.

Token batch size alone barely moved throughput.



A/B stayed flat; C/D/F jumped when max_num_seqs increased.

GROUP 3 | MLPerf

Extra vLLM tweaks were tested one at a time.

Chunked prefill won after the Run F batching setup.

1084.5

F batch
setup

1088.3

eager off

1193.3

chunked
prefill

1115.1

scheduler=2

Important: one-knob tests, not combined results.

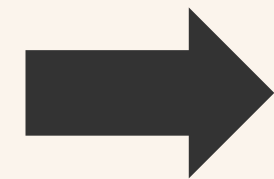
GROUP 3 | MLPerf

Final valid result: 4.54x more tokens/s.

Same model, same dataset, same Offline single-GPU rule.

262.591

baseline



1193.320

final

4.54x

gain

**Final config: 0.90 mem · 16384 tokens · 64
seqs · chunked prefill**

GROUP 3 | MLPerf

Appendix: Base to B confirm token window alone did not help.

Base

2048 tok · 8

seq · 0.70

262.591 t/s

A

4096 tok · 8

seq · 0.70

262.011 t/s

B

8192 tok · 8

seq · 0.70

262.330 t/s

🌀 **GROUP 3 | MLPerf**
**Appendix: C to F show sequence
concurrency scaling.**

C

8192 tok
16 seq · 0.80

462.105 t/s

D

8192 tok
32 seq · 0.80

755.355 t/s

E

16384 tok
32 seq · 0.90

752.100 t/s

F

16384 tok
64 seq · 0.90

1084.520 t/s

GROUP 3 | MLPerf

Appendix: G to I were one-knob tests after Run F.

G

enforce_eager
False

1088.250 t/s

H

chunked prefill
True

1193.320 t/s

I

scheduler
steps = 2

1115.070 t/s

NOT combined.

HEMELB OPTIMIZATIONS

杜武明光 - 113006214



THE IMPORTANCE OF SPACK



COMPILE LIBRARY WITHOUT PRIVILEGES

We can't install dependencies with Ubuntu's package manager, which requires root privileges. Spack allows us to install dependencies without



EASE OF INSTALLATION

Spack compile the library automatically, which can also be easily configured.



CONTAINERIZED ENVIRONMENT

Can create environments to create different build with different dependencies, compilers and features without affect other builds.

HEMELB

COMPILATION

CMake arguments

- -DHEMELB_GPU_BACKEND=HIP_ROCM
- -DCMAKE_CXX_COMPILER=hipcc
- -DAMDGPU_TARGETS="gfx1201"
- -DCMAKE_HIP_FLAGS="-O3 -fopenmp --offload-arch=gfx1201 -funsafe-math-optimizations -ffast-math -Rpass-analysis=kernel-resource-usage"
- -DHEMELB_USE_PARMETIS=ON
- -DCMAKE_HIP_ARCHITECTURES=gfx1201
- -DHEMELB_USE_SSE3=ON

HEMELB

OPTIMIZATION - REGISTER PRESSURE

```
/cuda_params.cu:1757:0: Function Name: _ZN6hemelb71GPU_CollideStream_mMidFluidCollision_mWallCollision_sBB_WallShearStress
/cuda_params.cu:1757:0:      TotalSGPRs: 91
/cuda_params.cu:1757:0:      VGPRs: 148
/cuda_params.cu:1757:0:      ScratchSize [bytes/lane]: 320
/cuda_params.cu:1757:0:      Dynamic Stack: False
/cuda_params.cu:1757:0:      Occupancy [waves/SIMD]: 9
/cuda_params.cu:1757:0:      SGPRs Spill: 0
/cuda_params.cu:1757:0:      VGPRs Spill: 0
/cuda_params.cu:1757:0:      LDS Size [bytes/block]: 0
```

Reduced VGPR down to around 50, but performance worsens.

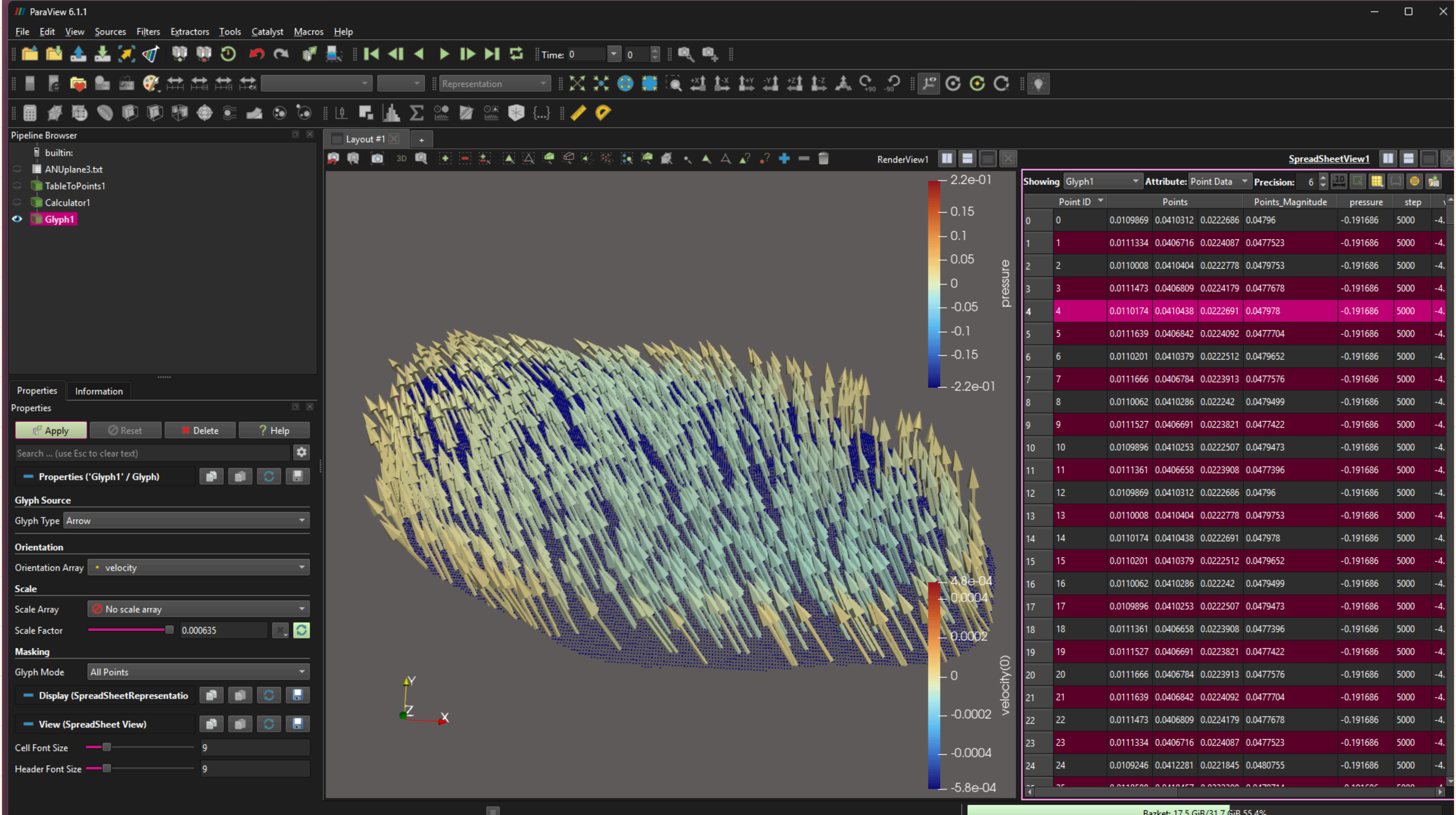
OPTIMIZATION - INPUT FILE

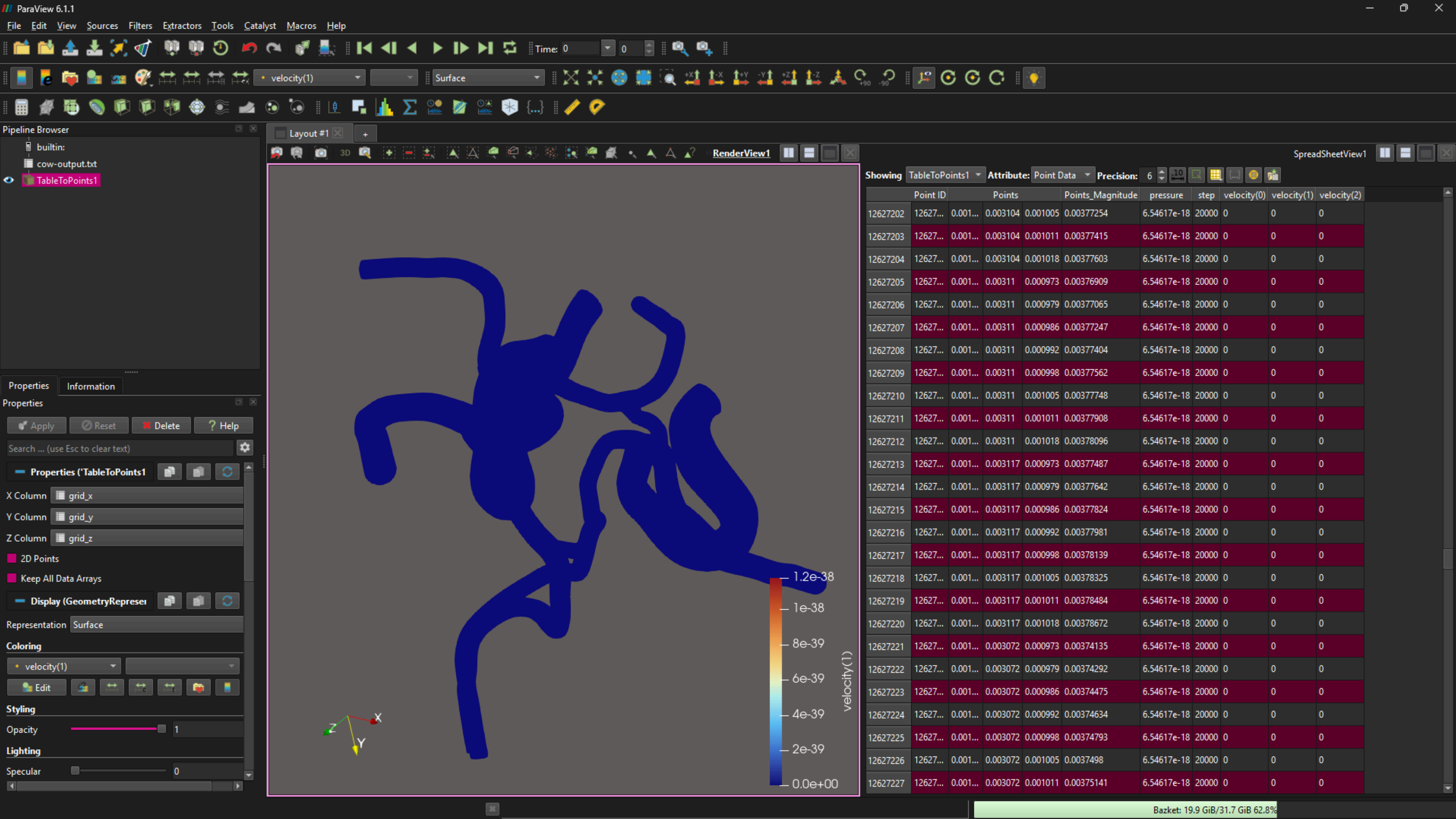
- Logging to files require moving data from GPU back to CPU, costing time.
- After removing non-necessary logging (only log what the problem ask for), performance improved.
- Increasing logging period worsens performance.

HEMELB VISUALIZATIONS

[HOME](#)[SERVICE](#)[ABOUT US](#)[CONTACT US](#)

- Compile hemeXtract
- Run: hemeXtract -X whole.dat -o file.txt
- Import the file into Paraview





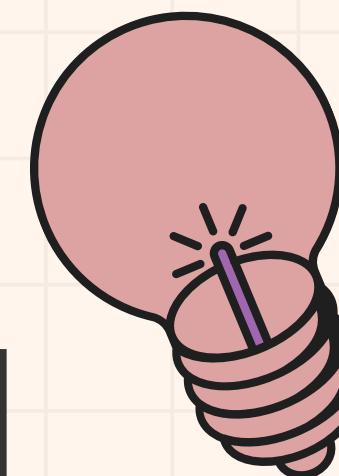
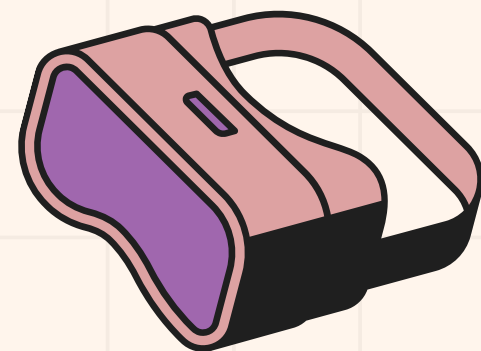


[HOME](#)

[SERVICE](#)

[ABOUT US](#)

[CONTACT US](#)



THANK YOU

